



به نام خدا

تمرین کامپیوتری چهارم درس طراحی کامپایلر

بهار ۱۴۰۵

ددلاین: انتهای سه‌شنبه، ۲۳ تیر

طراحان: فاطمه بلوک‌آذری - مهدیس میرزایی

فهرست مطالب

3	مقدمه
3	نکات کلی پیاده‌سازی
3	موارد پیاده‌سازی
5	دستورات تبدیل و اجرای کدها

مقدمه

در این فاز بخش‌های مربوط به تولید کد را به کامپایلر خود اضافه می‌کنید. در انتهای این فاز، کامپایلر شما به طور کامل پیاده‌سازی شده و برنامه‌های نوشته شده به زبان را به کد قابل اجرا توسط ماشین تبدیل می‌کند. پیاده‌سازی شما باید به ازای هر فایل ورودی، لیست bytecode های معادل آن را تولید کند. توجه کنید که شما برای تولید کد به اطلاعات جمع آوری شده در جدول علائم و اطلاعات مربوط به تایپ نودهای درخت AST نیاز دارید.

نکات کلی پیاده‌سازی

- برای متغیر boolean، اگر مقدار آن true باشد 1 و اگر مقدار آن false باشد 0 را در stack اضافه کنید.
- برای اضافه کردن مقادیر primitive به stack، از دستور ldc استفاده کنید.
- طول stack و locals را در متدها ۱۲۸ قرار دهید.
- برای انجام محاسبات روی Integer، مقادیر باید از نوع primitive یعنی int باشند.
- در اضافه کردن command ها به \n ها دقت داشته باشد تا در فایل jasmin ایجاد شده command ها پشت سر هم نباشند. همچنین هر command ای که اضافه می‌کنید به طور دقیق بررسی کنید که چه آرگومان‌هایی لازم دارد و چه مقداری را باز می‌گرداند؛ زیرا اگر اشتباهی رخ دهد debug کردن آن در فایل‌های jasmin کار دشواری است.
- یک نمونه سمپل به همراه تمپلیتی قرار داده شده است. توجه کنید می‌توانید اگر قسمتی از کد تمپلیت را می‌خواهید تغییر دهید و به نحوی که می‌خواهید انجام دهید. صرفاً برای کمک ویزیتور CodeGenerator قرار داده شده است.

موارد پیاده‌سازی

برای این قسمت نیز نمونه کدی در اختیار شما قرار می‌گیرد که شامل بدنه اصلی Code generation است و با تکمیل آن می‌توانید bytecode های مربوطه را تولید کنید.

در این قسمت تعدادی از کلاس‌ها و نحوه ایجاد کد برای آنها به صورت سطح بالا توضیح داده شده است که می‌توانید در پیاده‌سازی خود از آنها استفاده کنید.

تابع slotOf :

این تابع slot متغیرها را برمیگرداند. توجه کنید که slot صفر به صورت پیش‌فرض برای خود کلاس اصلی برنامه است و بعد از آن باید به ترتیب آرگومان‌های تابع اضافه شوند. برای دریافت یا ایجاد slot یک متغیر، کافیست تابع slotOf(varName) صدا زده شود که در صورت وجود نداشتن، آن متغیر به slots اضافه می‌شود و slot مربوط به آن داده شود. همچنین در ابتدای هر تابع slots خالی می‌شود.

دستورات شرطی :

دستورات مورد نیاز برای دستور شرطی را اضافه کنید. دقت کنید که در این بخش label های مناسب را تولید کرده و در ادامه استفاده کنید.

حلقه‌های تکرار :

ابتدا label های مورد نظر را اضافه کنید. سپس statement های آن را visit کنید

: BinaryExpression

ابتدا عملوند سمت چپ و سپس عملوند سمت راست visit شوند و مقادیر آنها روی stack قرار بگیرند. سپس عملگر مورد نظر اعمال شود.

: UnaryExpression

ابتدا عملوند آن visit شود و مقدار آن روی stack قرار بگیرد، سپس عملگر مورد نظر اعمال شود.

: Identifier

از slot متناسب با آن identifier باید با استفاده از aload مقدار آن load شود.

Mut و Ownership :

دقت داشته باشید این موارد در این مرحله تاثیری در خروجی ندارد. به عبارتی محدودیت‌ها و شرط‌هایی که اعمال می‌کردند در فاز قبلی بررسی شد.

: Structs و Modules

می‌توانید برای هر یک از آنها فایل `jasmin` جدا بسازید. برای قسمت `include` کردن باید تمامی ماژول‌هایی که `include` شده است را ابتدا فایل اضافه کنیم و آن‌ها را صدا کنیم. یک نمونه از نحوه پیاده‌سازی `include` در سمپل‌هایی که در اختیار شما قرار گرفته، وجود دارد. دقت داشته باشید توصیه می‌شود برای درک بهتر ساختار آن و نحوه پیاده‌سازی آن به نمونه کد `SimpleLang` در ایلرن مراجعه کنید. جهت تولید فایل‌های `class`. نهایی از شما انتظار نمی‌رود که فایل باینری را مستقیماً تولید کنید. برای این کار می‌توانید از اسمبلر [jasmin](#) استفاده کنید.

دستورات کاربردی `jasmin`

اضافه کردن به `ArrayList`

```
invokevirtual java/util/ArrayList/add(Ljava/lang/Object;)Z
```

گرفتن سائز `ArrayList`

```
invokevirtual java/util/ArrayList/size()I
```

تبدیل (cast) یک `Object` به یک کلاس `A`

```
checkcast A
```

دستورات تبدیل و اجرای کدها

کامپایل کردن فایل `java`. به فایل `class`:

```
javac -g *.java
```

اجرای فایل‌های `class`. (فایل `Main.class` باید اجرا شود):

```
java Main
```

تبدیل فایل `(.j)` `jasmin bytecode` به فایل `class`:

```
java -jar jasmin.jar *.j
```

تبدیل فایل class به java bytecode که خروجی هم در ترمینال نمایش داده شود:

```
javap -c -l A
```

تبدیل فایل class به jasmin bytecode که خروجی هم در ترمینال نمایش داده شود:

```
java -jar classFileAnalyzer.jar A.class
```